# Exploratory Test Session

Subject: Todos

## Charter

Identify capabilities and areas of potential instability of the "rest api todo list manager".
Identify documented and undocumented "rest api todo list manager" capabilities.
For each capability create a script or small program to demonstrate the capability.
Exercise each capability identified with data typical to the intended use of the application.

## Build

runTodoManagerRestAPI-1.5.5.jar

## Area

Todos – main functions and capabilities, mostly JSON with a short XML check at the end

## Tester

Yanick Wiaderek  261183242 yanick.wiaderek@mail.mcgill.ca

## Environment

macOS, VS Code terminal, curl

## Start

6:33 PM 10/17/2026

## Session Duration

42 minutes

## Files

- Session1_TodoCapabilities.sh (capability script)
- BUG-05 to BUG-10 (bug summaries)
- Screenshots of individual bugs, reproduced after the session (below every ##BUG line)

# TEST NOTES

I made sure to restart the API before starting to ensure I would be working with the default data. For the testing data I went with a clothing shopping list. The selection may be a bit niche but I personally write lists in todo format when I go shopping.

6:33 PM
Test: GET /todos to see what data exists by default and what a todo looks like.
Result: 200 with two todos, "scan paperwork" and "file paperwork". doneStatus comes back as the string "false", even though the docs say it is a BOOLEAN. #ISSUE

6:34 PM
Test: HEAD /todos to check it returns headers only.
Result: 200 with headers and no body. As expected.

6:35 PM
Test: GET /todos/1 to retrieve a single todo by id.
Result: 200 with the right todo, but it is still wrapped in a "todos" list even though I asked for one. Not wrong, just something a client has to handle.

6:36 PM
Test: GET /todos/999 to see how the API handles a todo that doesn't exist.
Result: 404 "Could not find an instance with todos/999". Clear and correct.

6:36 PM
Test: HEAD on /todos/1 and /todos/999 to check HEAD matches GET.
Result: 200 and 404, no body in either. Consistent with GET.

6:39 PM
Test: Create a todo with typical data: "Buy clothing", description "Pants, sHoes", doneStatus false.
Result: 201, id 3, Location header todos/3. Works. I sent doneStatus as a real boolean but it came back as the string "false" again.

6:40 PM
Test: GET the new todo to confirm it was saved. (First attempt had a typo that sent /todos/ with a trailing slash.)
Result: The typo request gave a 404 with an HTML content type and no message, unlike every other 404 so far. #ISSUE  The corrected GET /todos/3 returned "Buy clothing" with all values saved.
Next: test the validation rules listed in the docs.

6:43 PM
Test: Create "Buy a winter jacket" with doneStatus as the string "true", to check type validation.

Result: 400 "doneStatus should be BOOLEAN". Same on a second try. Strange, since the API refuses string booleans but sends them back as strings itself. A client that sends back what it received would get an error. #ISSUE

6:44 PM
Test: Create "Buy running shoes" with an extra field "size" that isn't in the model, to see if unknown fields are ignored.
Result: 400 "Could not find field: size". Unknown fields are rejected, not ignored.

6:45 PM
Test: Create "Buy underwear" with only a title, to see what defaults are used.
Result: 201, defaults are doneStatus false and an empty description. But the id is 7, not 4. The three rejected requests in between seem to have used up ids 4, 5 and 6. #ISSUE
Next: check this right away with another create.

6:47 PM
Test: Create "Buy a scarf" with no failed requests in between, as a control for the id gap.
Result: 201, id 8. No gap this time, which supports the idea that failed requests use up ids.

6:48 PM
Test: Create a todo with an empty title. The docs say title "can not be empty".
Result: 400 "title : can not be empty". Matches the docs.

6:49 PM
Test: Create "Buy a belt" with my own id (50), to see if clients can choose ids.
Result: 400 "Not allowed to create with id". Ids are generated by the server only.

6:51 PM
Test: Create a todo with only a description ("Jeans, size 32") and no title. The docs say title is mandatory.
Result: 400 "title : field is mandatory". Same on a second try. A different message from the empty title case.

6:54 PM
Test: Create "Buy gloves" after those failures, to confirm the id gap.
Result: 201, id 12, so ids 9 to 11 were skipped. Confirms failed creates use up ids. ##BUG (BUG-10)
Interesting detail: four failed requests but only three ids skipped. Will look into it after the session.

```
yanickwiaderek@Yanicks-MacBook-Air ECSE429-PartA-main % clear
curl -s -X POST $B/todos -H "$J" -d '{"title":"Buy clothing","description":"Pants, Shoes","doneStatus":false}'; echo
curl -s -X POST $B/todos -H "$J" -d '{"title":""}'; echo
curl -s -X POST $B/todos -H "$J" -d '{"title":"Buy gloves"}'; echo
{"id":"6","title":"Buy clothing","doneStatus":"false","description":"Pants, Shoes"}
{"errorMessages":["Failed Validation: title : can not be empty"]}
{"id":"8","title":"Buy gloves","doneStatus":"false","description":""}
```

*Reproduced after the session (Oct 18, ~8:30 PM EDT; any Date header shown is in GMT)*

6:56 PM
Test: POST to todo 3 with only doneStatus true, to check partial updates.
Result: 200. Only doneStatus changed, title and description stayed. Works.

6:56 PM
Test: PUT to todo 3 with only a new title, "Buy clothing for winter". The docs describe PUT as amending the fields you send, the same as POST.
Result: 200. The title changed, but doneStatus went back to false and the description "Pants, sHoes" was erased. PUT silently deletes data you didn't send. ##BUG (BUG-05)

```
yanickwiaderek@Yanicks-MacBook-Air ECSE429-PartA-main % clear
curl -s -X POST $B/todos/3 -H "$J" -d '{"doneStatus":true}'; echo
curl -s -X PUT $B/todos/3 -H "$J" -d '{"title":"Buy clothing for winter"}'; echo
curl -s $B/todos/3; echo
{"id":"3","title":"Buy clothing","doneStatus":"true","description":"Pants, sHoes"}
{"id":"3","title":"Buy clothing for winter","doneStatus":"false","description":""}
{"todos":[{"id":"3","title":"Buy clothing for winter","doneStatus":"false","description":""}]}
```

*Reproduced after the session (Oct 18, ~8:30 PM EDT; any Date header shown is in GMT)*

6:56 PM
Test: GET /todos/3 to check whether the erased values were really saved that way.
Result: Yes. The todo now has doneStatus false and an empty description.

6:56 PM
Test: PUT to todo 3 with only a description ("fishing gear"), to see if PUT needs a title.
Result: 400 "title : field is mandatory". PUT always needs a title, POST doesn't.

6:56 PM
Test: POST and PUT to todo 999, to compare how each handles a todo that doesn't exist.
Result: Both 404, but with different messages: "No such todo entity instance with GUID or ID 999 found" and "Invalid GUID for 999 entity todo". That's three different wordings for "not found" now. #ISSUE

6:59 PM
Test: OPTIONS /todos/1 to find out which methods are supported. Not in the docs.
Result: 200, Allow header lists OPTIONS, GET, HEAD, POST, PUT, DELETE. Useful and undocumented.

6:59 PM
Test: Filter with a field that doesn't exist (?size=10).
Result: 200 with every todo, no error. Inconsistent with POST, which rejected "size" earlier. #ISSUE

6:59 PM
Test: Delete the scarf (id 8), GET it, then delete it again.

Result: 200, then 404, then 404. Works as expected.

6:59 PM
Test: PATCH on todo 1 and DELETE on the whole /todos collection, which the docs don't list.
Result: Both 405. Good, deleting every todo at once would be dangerous.

6:59 PM
Test: Filter by title "Buy gloves", then by doneStatus=false.
Result: The title filter returned only the gloves. doneStatus=false returned all unfinished todos, with the scarf gone. Filters work.
Next: relationships. Olivia found bugs on the project side with fake ids, so I want to see if todos have the same problem.

7:01 PM
Test: Link the gloves (12) to category 1.
Result: 201. Link created.

7:01 PM
Test: GET the projects of todo 999, which doesn't exist.
Result: 200 instead of 404, with the project "Office Work" listed twice. ##BUG (BUG-06)




*Reproduced after the session (Oct 18, ~8:30 PM EDT; any Date header shown is in GMT)*

7:01 PM
Test: Link the gloves to project 1.
Result: 201. Link created.

7:01 PM
Test: GET category 1's todos, to see if the gloves link shows up on the category side.
Result: Empty list, even though the gloves and todo 1 are both linked to category 1. The link only exists on the todo side. ##BUG (BUG-08)

```
yanickwiaderek@Yanicks-MacBook-Air ECSE429-PartA-main % clear
curl -s -o /dev/null -w "link: %{http_code}\n" -X POST $B/todos/5/categories -H "$J" -d '{"id":"1"}'
curl -s $B/todos/5/categories; echo
curl -i $B/categories/1/todos; echo
link: 201
{"categories":[{"id":"1","title":"Office","description":""}]}
HTTP/1.1 200 OK
Date: Mon, 19 Oct 2026 00:36:25 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"todos":[]}
```

*Reproduced after the session (Oct 18, ~8:30 PM EDT; any Date header shown is in GMT)*

7:01 PM
Test: GET the categories of todo 999, which doesn't exist.
Result: 200 instead of 404, with "Office" listed twice. Same problem as the projects one. ##BUG (BUG-07)

```
yanickwiaderek@Yanicks-MacBook-Air ECSE429-PartA-main % clear
curl -i $B/todos/999/categories; echo
HTTP/1.1 200 OK
Date: Mon, 19 Oct 2026 00:39:05 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"categories":[{"id":"1","title":"Office","description":""},{"id":"1","title":"Office","description":""}]}
```

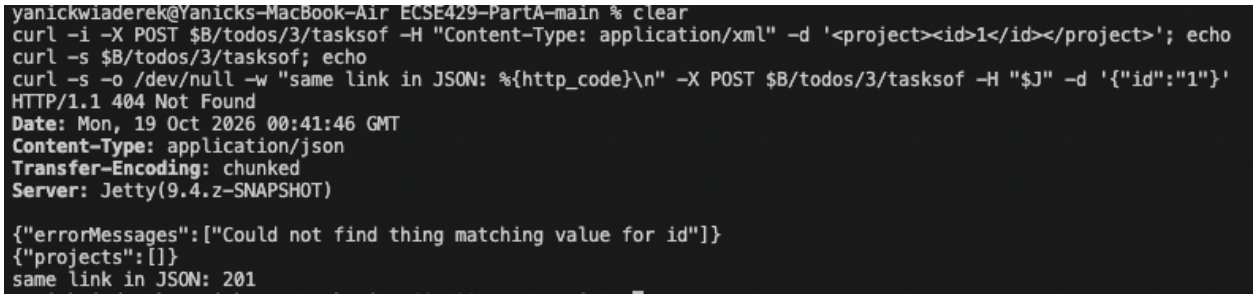
*Reproduced after the session (Oct 18, ~8:30 PM EDT; any Date header shown is in GMT)*

7:01 PM
Test: Send a project title instead of an id to /todos/12/tasksof, to see what happens.
Result: 201. It created a brand new project "Winter wardrobe" and linked it to the gloves in one step. Not documented anywhere.

7:01 PM
Test: GET project 1's tasks, to see if the gloves link shows up on the project side.
Result: The gloves are there. The todo-project link works both ways, unlike categories.

7:01 PM
Test: Remove the gloves from category 1.
Result: 200. Link removed.
Next: short XML check, focusing on relationships.

7:06 PM
Test: Link todo 3 to project 1 using an XML body instead of JSON.
Result: 404 "Could not find thing matching value for id". The same request in JSON works. ##BUG (BUG-09)




*Reproduced after the session (Oct 18, ~8:30 PM EDT; any Date header shown is in GMT)*

7:06 PM
Test: Delete the gloves, which are linked to projects 1 and 2, to check for side effects.
Result: 200. Checked below.

7:06 PM
Test: Send broken JSON (an extra comma) when creating "Buy boots".
Result: 400, but the response is marked as HTML and shows the name of an internal Java exception. Users shouldn't see that. #ISSUE

7:06 PM
Test: GET project 1's tasks after deleting the gloves.
Result: The gloves are gone and todos 1 and 2 are untouched. No side effects.

7:06 PM
Test: Create "Buy a hat" using XML.
Result: 201 with an XML response, id 13. Works. The broken JSON request didn't use up an id.

7:06 PM
Test: GET project 2 ("Winter wardrobe") after deleting the gloves.
Result: Still exists, with the gloves removed from its tasks. Deleting a todo removes its links but not the projects it was linked to. Makes sense.

7:10 PM
Test: GET todo 3's projects, to check the failed XML link didn't half-save.
Result: Empty list. Nothing was changed.

7:12 PM
Test: Link the hat to category 1 using XML, to see if the XML bug also affects categories.
Result: Same 404. Ran it again to be sure. The XML problem affects categories too.

7:15 PM  Ending session.

## Post Testing

After the session, I wanted to understand why four failed requests only resulted in three skipped IDs. To investigate this, I tested each type of invalid request individually, placing it between two valid POST requests to see whether an ID was skipped.

I found that requests with invalid field values (empty title, missing title, doneStatus as a string, unknown field) would consume an ID even when rejected. However, requests that included a manually assigned ID or malformed JSON did not.

This explains what happened during the session. The empty title and two missing title requests accounted for the three skipped IDs, while the request where I assigned ID 50 did not consume one.

## Discovered Bugs

1. PUT /todos/:id resets any field not in the payload to its default (BUG-05)
2. GET /todos/:id/tasksof with a nonexistent id returns 200 and unrelated (repeated) projects instead of 404 (BUG-06)
3. Same thing on GET /todos/:id/categories (BUG-07)
4. Todo -> category links aren't visible from /categories/:id/todos, so the relationship is one-way (BUG-08)
5. POST to tasksof/categories with an XML body returns 404, while the same request in JSON works (BUG-09)
6. Failed creates still increment the id counter (BUG-10)

Bugs closed: none.

## Summary Findings

- The basic todo operations all work in JSON: creating, reading, HEAD, updating with POST and deleting. In XML I only tried creating a todo, which worked.
- Validation mostly follows the docs. A title is required and can't be empty, doneStatus has to be a real boolean, and the API rejects client-chosen ids and fields that aren't in the model.
- PUT doesn't work the way the docs describe. Instead of amending the fields you send, it replaces the whole todo and always needs a title.
- Filtering with query parameters works for both title and doneStatus.
- Deleting a todo cleans up its links to projects without deleting the projects themselves, so I didn't see any side effects there.
- Links between todos and projects show up on both sides, but links between todos and categories only show up on the todo side.
- Relationships can only be created with JSON. The same request in XML fails.
- The relationship endpoints don't check whether the todo exists. A fake id gets back real projects or categories that belong to other todos, sometimes repeated.

## Areas of risk

- A client using a stale or wrong todo id gets back someone else's projects/categories with a 200, so it has no way to know something went wrong.
- Referential integrity problem with categories, the two sides can disagree.
- XML clients can't create relationships at all.
- Data loss risk: any client using PUT for a small edit wipes description and doneStatus.
- Error handling is inconsistent: three different "not found" messages, some errors come back as text/html, and malformed JSON leaks the internal Gson exception.
- doneStatus is returned as a string but must be sent as a boolean, so round-tripping a todo fails.

## New Testing Ideas

1. Create the link from the category side (POST /categories/:id/todos) and check whether the todo sees it, to find out if BUG-08 only goes one way.
2. Delete a project or category that still has todos linked to it and check what happens to those todos.
3. Link the same todo to the same project twice. Does it create a duplicate or return an error?
4. Send a PUT with every field included, to see whether PUT only causes data loss when fields are left out.
5. Do a full session in XML, mainly to check PUT behaviour and whether error responses come back in XML.
6. Send a JSON body with Content-Type: application/xml (and the other way around) to see how the API handles mismatched headers.

## Capabilties

Documented:
- GET and HEAD on /todos and /todos/:id
- POST /todos (create), POST and PUT /todos/:id (update), DELETE /todos/:id
- Filtering through query params (?doneStatus=false, ?title=...)
- GET, POST, DELETE for todo-project (tasksof) and todo-category relationships
- JSON and XML payloads

Undocumented:
- OPTIONS returns the allowed methods in the Allow header
- POST /todos/:id/tasksof with a title instead of an id creates a new project and links it
- PATCH on a todo and DELETE on the whole collection return 405

All capabilities are scripted in Session1_TodoCapabilities.sh.